

Chapter 5 Pipelines

You will never walk again, but you will fly!

— Three-Eyed Raven

In [Chapter 4](#), you learned how to build predictive models using the high-level functions Spark provides and well-known R packages that work well together with Spark. You learned about supervised methods first and finished the chapter with an unsupervised method over raw text.

In this chapter, we dive into Spark Pipelines, which is the engine that powers the features we demonstrated in [Chapter 5](#). So, for instance, when you invoke an `MLlib` function via the formula interface in R—for example, `ml_logistic_regression(cars, am ~ .)`—a *pipeline* is constructed for you under the hood. Therefore, `Pipelines`((("pipelines", "purpose of"))) also allow you to make use of advanced data processing and modeling workflows. In addition, a pipeline also facilitates collaboration across data science and engineering teams by allowing you to *deploy* pipelines into production systems, web applications, mobile applications, and so on.

This chapter also happens to be the last chapter that encourages using your local computer as a Spark cluster. You are just one chapter away from getting properly introduced to cluster computing and beginning to perform data science or machine learning that can scale to the most demanding computation problems.

5.1 Overview

The building blocks of pipelines are objects called *transformers* and *estimators*, which are collectively referred to as *pipeline stages*. A *transformer* can be used to apply transformations to a `DataFrame` and return another `DataFrame`; the resulting `DataFrame` often comprises the original `DataFrame` with new columns appended to it. An *estimator*, on the other hand, can be used to create a transformer giving some training data. Consider the following example to illustrate this relationship: a “center and scale” estimator can learn the mean and standard deviation of some data and store the statistics in a resulting transformer object; this transformer can then be used to normalize the data that it was trained on and also any new, yet unseen, data.

Here is an example of how to define an estimator:

```
library(sparklyr)
library(dplyr)

sc <- spark_connect(master = "local", version = "2.3")

scaler <- ft_standard_scaler(
  sc,
  input_col = "features",
  output_col = "features_scaled",
  with_mean = TRUE)

scaler
```

```
StandardScaler (Estimator)
<standard_scaler_7f6d46f452a1>
(Parameters -- Column Names)
  input_col: features
  output_col: features_scaled
(Parameters)
  with_mean: TRUE
  with_std: TRUE
```

We can now create some data (for which we know the mean and standard deviation) and then fit our scaling model to it using the `ml_fit()` function:

```
df <- copy_to(sc, data.frame(value = rnorm(100000))) %>%
  ft_vector_assembler(input_cols = "value", output_col = "features")

scaler_model <- ml_fit(scaler, df)
scaler_model
```

```
StandardScalerModel (Transformer)
```

```
<standard_scaler_7f6d46f452a1>
```

```
(Parameters -- Column Names)
```

```
input_col: features
```

```
output_col: features_scaled
```

```
(Transformer Info)
```

```
mean: num 0.00421
```

```
std: num 0.999
```

Note: In Spark ML, many algorithms and feature transformers require that the input be a vector column. The function `ft_vector_assembler()` performs this task. You can also use the function to initialize a transformer to be used in a pipeline.

We see that the mean and standard deviation are very close to 0 and 1, respectively, which is what we expect. We then can use the transformer to *transform* a DataFrame, using the `ml_transform()` function:

```
scaler_model %>%
  ml_transform(df) %>%
  glimpse()
```

```
Observations: ??
```

```
Variables: 3
```

```
Database: spark_connection
```

```
$ value <dbl> 0.75373300, -0.84207731, 0.59365113, -...
```

```
$ features <list> [0.753733, -0.8420773, 0.5936511, -0....
```

```
$ features_scaled <list> [0.7502211, -0.8470762, 0.58999, -0.4...
```

Now that you've seen basic examples of estimators and transformers, we can move on to pipelines.

5.2 Creation

A *pipeline* is simply a sequence of transformers and estimators, and a *pipeline model* is a pipeline that has been trained on data so all of its components have been converted to transformers.

There are a couple of ways to construct a pipeline in `sparklyr`, both of which use the `ml_pipeline()` function.

We can initialize an empty pipeline with `ml_pipeline(sc)` and append stages to it:

```
ml_pipeline(sc) %>%
  ft_standard_scaler(
    input_col = "features",
    output_col = "features_scaled",
    with_mean = TRUE)
```

```
Pipeline (Estimator) with 1 stage
<pipeline_7f6d6a6a38ee>
  Stages
|--1 StandardScaler (Estimator)
|   <standard_scaler_7f6d63bfc7d6>
|   (Parameters -- Column Names)
|     input_col: features
|     output_col: features_scaled
|   (Parameters)
|     with_mean: TRUE
|     with_std: TRUE
```

Alternatively, we can pass stages directly to `ml_pipeline()` :

```
pipeline <- ml_pipeline(scaler)
```

We fit a pipeline as we would fit an estimator:

```
pipeline_model <- ml_fit(pipeline, df)
pipeline_model
```

PipelineModel (Transformer) with 1 stage

```
<pipeline_7f6d64df6e45>
  Stages
|--1 StandardScalerModel (Transformer)
|   <standard_scaler_7f6d46f452a1>
|   (Parameters -- Column Names)
|   input_col: features
|   output_col: features_scaled
|   (Transformer Info)
|   mean:  num 0.00421
|   std:   num 0.999
```

```
pipeline
```

Note: As a result of the design of Spark ML, pipelines are always estimator objects, even if they comprise only transformers. This means that if you have a pipeline with only transformers, you still need to call `ml_fit()` on it to obtain a transformer. The “fitting” procedure in this case wouldn’t actually modify any of the transformers.

5.3 Use Cases

Now that you have an understanding of the rudimentary concepts for ML Pipelines, let’s apply them to the predictive modeling problem from the previous chapter in which we are trying to predict whether people are currently employed by looking at their profiles. Our starting point is the `okc_train` `DataFrame` with the relevant columns.

```
okc_train <- spark_read_parquet(sc, "data/okc-train.parquet")

okc_train <- okc_train %>%
  select(not_working, age, sex, drinks, drugs, essay1:essay9, essay_length)
```

We first exhibit the pipeline, which includes feature engineering and modeling steps, and then walk through it:

```
pipeline <- ml_pipeline(sc) %>%
  ft_string_indexer(input_col = "sex", output_col = "sex_indexed") %>%
  ft_string_indexer(input_col = "drinks", output_col = "drinks_indexed") %>%
  ft_string_indexer(input_col = "drugs", output_col = "drugs_indexed") %>%
  ft_one_hot_encoder_estimator(
    input_cols = c("sex_indexed", "drinks_indexed", "drugs_indexed"),
    output_cols = c("sex_encoded", "drinks_encoded", "drugs_encoded")
  ) %>%
  ft_vector_assembler(
    input_cols = c("age", "sex_encoded", "drinks_encoded",
                  "drugs_encoded", "essay_length"),
    output_col = "features"
  ) %>%
  ft_standard_scaler(input_col = "features", output_col = "features_scaled",
                    with_mean = TRUE) %>%
  ml_logistic_regression(features_col = "features_scaled",
                        label_col = "not_working")
```

The first three stages index the `sex`, `drinks`, and `drugs` columns, which are characters, into numeric indices via `ft_string_indexer()`. This is necessary for the `ft_one_hot_encoder_estimator()` that comes next, which requires numeric column inputs. When all of our predictor variables are of numeric type (recall that `age` is numeric already), we can create our features vector using `ft_vector_assembler()`, which concatenates all of its inputs together into one column of vectors. We can then use `ft_standard_scaler()` to normalize all elements of the features column (including the one-hot-encoded 0/1 values of the categorical variables), and finally apply a logistic regression via `ml_logistic_regression()`.

During prototyping, you might want to execute these transformations *eagerly* on a small subset of the data, by passing the DataFrame to the `ft_` and `ml_` functions, and inspecting the transformed DataFrame. The immediate feedback allows for rapid iteration of ideas; when you have arrived at the desired processing steps, you can roll them up into a pipeline. For example, you can do the following:

```
okc_train %>%
  ft_string_indexer("sex", "sex_indexed") %>%
  select(sex_indexed)
```

```
# Source: spark<?> [?? x 1]
```

```
  sex_indexed
      <dbl>
1           0
2           0
3           1
4           0
5           1
6           0
7           0
8           1
9           1
10          0
# ... with more rows
```

After you have found the appropriate transformations for your dataset, you can replace the DataFrame input with `ml_pipeline(sc)`, and the result will be a pipeline that you can apply to any DataFrame with the appropriate schema. In the next section, we'll see how pipelines can make it easier for us to test different model specifications.

5.3.1 Hyperparameter Tuning

Going back to the pipeline we have created earlier, we can use `ml_cross_validator()` to perform the cross-validation workflow we demonstrated in the previous chapter and easily test different hyperparameter combinations. In this example, we test whether centering the variables improves predictions together with various regularization values for the logistic regression. We define the cross-validator as follows:

```

cv <- ml_cross_validator(
  sc,
  estimator = pipeline,
  estimator_param_maps = list(
    standard_scaler = list(with_mean = c(TRUE, FALSE)),
    logistic_regression = list(
      elastic_net_param = c(0.25, 0.75),
      reg_param = c(1e-2, 1e-3)
    )
  ),
  evaluator = ml_binary_classification_evaluator(sc, label_col = "not_working"),
  num_folds = 10)

```

The `estimator` argument is simply the estimator that we want to tune, and in this case it is the `pipeline` that we defined. We provide the hyperparameter values we are interested in via the `estimator_param_maps` parameter, which takes a nested named list. The names at the first level correspond to UIDs, which are unique identifiers associated with each pipeline stage object, of the stages we want to tune (if a partial UID is provided, `sparklyr` will attempt to match it to a pipeline stage), and the names at the second level correspond to parameters of each stage. In the preceding snippet, we are specifying that we want to test the following:

Standard scaler

The values `TRUE` and `FALSE` for `with_mean`, which denotes whether predictor values are centered.

Logistic regression

The values `0.25` and `0.75` for α , and the values `1e-2` and `1e-3` for λ .

We expect this to give rise to $2 \times 2 \times 2 = 8$ hyperparameter combinations, which we can confirm by printing the `cv` object:

```
cv
```

```

CrossValidator (Estimator)
<cross_validator_d5676ac6f5>
(Parameters -- Tuning)
  estimator: Pipeline
    <pipeline_d563b0cba31>
  evaluator: BinaryClassificationEvaluator
    <binary_classification_evaluator_d561d90b53d>
  with metric areaUnderROC
  num_folds: 10
[Tuned over 8 hyperparameter sets]

```

As with any other estimator, we can fit the cross-validator by using `ml_fit()`

```
cv_model <- ml_fit(cv, okc_train)
```

and then inspect the results:

```

ml_validation_metrics(cv_model) %>%
  arrange(-areaUnderROC)

```

	areaUnderROC	elastic_net_param_1	reg_param_1	with_mean_2
1	0.7722700	0.75	0.001	TRUE
2	0.7718431	0.75	0.010	FALSE
3	0.7718350	0.75	0.010	TRUE
4	0.7717677	0.25	0.001	TRUE
5	0.7716070	0.25	0.010	TRUE
6	0.7715972	0.25	0.010	FALSE
7	0.7713816	0.75	0.001	FALSE
8	0.7703913	0.25	0.001	FALSE

Now that we have seen the pipelines API in action, let's talk more formally about how they behave in various contexts.

5.4 Operating Modes

By now, you have likely noticed that the pipeline stage functions, such as `ft_string_indexer()` and `ml_logistic_regression()`, return different types of objects depending on the first argument passed to them. Table 5.1 presents the full pattern.

TABLE 5.1: Operating modes in machine learning functions.

First.argument	Returns	Example
Spark connection	Estimator or transformer object	<code>ft_string_indexer(sc)</code>
Pipeline	Pipeline	<code>ml_pipeline(sc) %>% ft_string_indexer()</code>
Data frame, without formula	Data frame	<code>ft_string_indexer(iris, "Species", "indexed")</code>
Data frame, with formula	sparklyr ML model object	<code>ml_logistic_regression(iris, Species ~ .)</code>

These functions are implemented using [S3](#), which is the most popular object-oriented programming paradigm provided by R. For our purposes, it suffices to know that the behavior of an `ml_` or `ft_` function is dictated by the class of the first argument provided. This allows us to provide a wide range of features without introducing additional function names. We now can summarize the behavior of these functions:

- If a Spark connection is provided, the function returns a transformer or estimator object, which can be utilized directly using `ml_fit()` or `ml_transform()` or be included in a pipeline.
- If a pipeline is provided, the function returns a pipeline object with the stage appended to it.
- If a `DataFrame` is provided to a feature transformer function (those with prefix `ft_`), or an ML algorithm without also providing a formula, the function instantiates the pipeline stage object, fits it to the data if necessary (if the stage is an estimator), and then transforms the `DataFrame` returning a `DataFrame`.
- If a `DataFrame` and a formula are provided to an ML algorithm that supports the formula interface, `sparklyr` builds a pipeline model under the hood and returns an ML model object that contains additional metadata information.

The formula interface approach is what we studied in [Chapter 5](#), and this is what we recommend users new to Spark start with, since its syntax is similar to existing R modeling packages and abstracts away some Spark ML peculiarities. However, to take advantage of the full power of Spark ML and leverage pipelines for workflow organization and interoperability, it is worthwhile to learn the ML Pipelines API.

With the basics of pipelines down, we are now ready to discuss the collaboration and model deployment aspects hinted at in the introduction of this chapter.

5.5 Interoperability

One of the most powerful aspects of pipelines is that they can be serialized to disk and are fully interoperable with the other Spark APIs such as Python and Scala. This means that you can easily share them among users of Spark working in different languages, which might include other data scientists, data engineers, and deployment engineers. To save a pipeline model, call `ml_save()` and provide a path:

```
model_dir <- file.path("spark_model")
ml_save(cv_model$best_model, model_dir, overwrite = TRUE)
```

Model successfully saved.

Let's take a look at the directory to which we just wrote:

```
list.dirs(model_dir, full.names = FALSE) %>%
  head(10)
```

```
[1] ""
[2] "metadata"
[3] "stages"
[4] "stages/0_string_indexer_5b42c72817b"
[5] "stages/0_string_indexer_5b42c72817b/data"
[6] "stages/0_string_indexer_5b42c72817b/metadata"
[7] "stages/1_string_indexer_5b423192b89f"
[8] "stages/1_string_indexer_5b423192b89f/data"
[9] "stages/1_string_indexer_5b423192b89f/metadata"
[10] "stages/2_string_indexer_5b421796e826"
```

We can dive into a couple of the files to see what type of data was saved:

```
spark_read_json(sc, file.path(
  file.path(dir(file.path(model_dir, "stages"),
    pattern = "1_string_indexer.*",
    full.names = TRUE), "metadata")
)) %>%
  glimpse()
```

Observations: ??

Variables: 5

Database: spark_connection

```
$ class      <chr> "org.apache.spark.ml.feature.StringIndexerModel"
$ paramMap   <list> [["error", "drinks", "drinks_indexed", "frequencyDesc"]]
$ sparkVersion <chr> "2.3.2"
$ timestamp   <dbl> 1.561763e+12
$ uid        <chr> "string_indexer_ce05afa9899"
```

```
spark_read_parquet(sc, file.path(
  file.path(dir(file.path(model_dir, "stages"),
    pattern = "6_logistic_regression.*",
    full.names = TRUE), "data")
))
```

```
# Source: spark<data> [?? x 5]
  numClasses numFeatures interceptVector coefficientMatr... isMultinomial
      <int>      <int> <list>           <list>           <lg1>
1           2        12 <dbl [1]>      <-1.27950828662... FALSE
```

We see that quite a bit of information has been exported, from the SQL statement in the `dplyr` transformer to the fitted coefficient estimates of the logistic regression. We can then (in a new Spark session) reconstruct the model by using `ml_load()` :

```
model_reload <- ml_load(sc, model_dir)
```

Let's see if we can retrieve the logistic regression stage from this pipeline model:

```
ml_stage(model_reload, "logistic_regression")
```

```
LogisticRegressionModel (Transformer)
<logistic_regression_5b423b539d0f>
(Parameters -- Column Names)
  features_col: features_scaled
  label_col: not_working
  prediction_col: prediction
  probability_col: probability
  raw_prediction_col: rawPrediction
(Transformer Info)
  coefficient_matrix: num [1, 1:12] -1.2795 -0.0915 0 0.126 -0.0324 ...
  coefficients: num [1:12] -1.2795 -0.0915 0 0.126 -0.0324 ...
  intercept: num -2.79
  intercept_vector: num -2.79
  num_classes: int 2
  num_features: int 12
  threshold: num 0.5
  thresholds: num [1:2] 0.5 0.5
```

Note that the exported JSON and parquet files are agnostic of the API that exported them. This means that in a multilingual machine learning engineering team, you can pick up a data preprocessing pipeline from a data engineer working in Python, build a prediction model on top

of it, and then hand off the final pipeline to a deployment engineering working in Scala. In the next section, we discuss deployment of models in more detail.

Note: When `ml_save()` is called for `sparklyr` ML models (created using the formula interface), the associated pipeline model is saved, but any `sparklyr`-specific metadata, such as index labels, is not. In other words, saving a `sparklyr ml_model` object and then loading it will yield a pipeline model object, as if you created it via the ML Pipelines API. This behavior is required to use pipelines with other programming languages.

Before we move on to discuss how to run pipelines in production, make sure you disconnect from Spark:

```
spark_disconnect(sc)
```

That way, we can start from a brand new environment, which is also expected when you are deploying pipelines to production.

5.6 Deployment

What we have just demonstrated bears emphasizing: by collaborating within the framework of ML Pipelines, we reduce friction among different personas in a data science team. In particular, we cut down on the time from modeling to deployment.

In many cases, a data science project does not end with just a slide deck with insights and recommendations. Instead, the business problem at hand might require scoring new data points on a schedule or on-demand in real time. For example, a bank might want to evaluate its mortgage portfolio risk nightly or provide instant decisions on credit card applications. This process of taking a model and turning it into a service that others can consume is usually referred to as *deployment* or *productionization*. Historically, there was a large gap between the analyst who built the model and the engineer who deployed it: the former might work in R and develop extensive documentation on the scoring mechanism, so that the latter can reimplement the model in C++ or Java. This practice, which might easily take months in some organizations, is less prevalent today, but is almost always unnecessary in Spark ML workflows.

The aforementioned nightly portfolio risk and credit application scoring examples represent two modes of ML deployment known as *batch* and *real time*. Loosely, batch processing implies processing many records at the same time, and that execution time is not important as long it

is reasonable (often on the scale of minutes to hours). On the other hand, real-time processing implies scoring one or a few records at a time, but the latency is crucial (on the scale of <1 second). Let's turn now to see how we can take our `okCupid` pipeline model to "production".

5.6.1 Batch Scoring

For both scoring methods, batch and real time, we will expose our model as web services, in the form of an API over the Hypertext Transfer Protocol (HTTP). This is the primary medium over which software communicates. By providing an API, other services or end users can utilize our model without any knowledge of R or Spark. The `plumber` R package enables us to do this very easily by annotating our prediction function.

You will need make sure that `plumber`, `callr`, and the `httr` package are installed by running the following:

```
install.packages(c("plumber", "callr", "httr"))
```

The `callr` package provides support to run R code in separate R sessions; it is not strictly required, but we will use it to start a web service in the background. The `httr` package allows us to use web APIs from R.

In the batch scoring use case, we simply initiate a Spark connection and load the saved model. Save the following script as *plumber/spark-plumber.R*:

```

library(sparklyr)

sc <- spark_connect(master = "local", version = "2.3")

spark_model <- ml_load(sc, "spark_model")

## @post /predict
score_spark <- function(age, sex, drinks, drugs, essay_length) {
  new_data <- data.frame(
    age = age,
    sex = sex,
    drinks = drinks,
    drugs = drugs,
    essay_length = essay_length,
    stringsAsFactors = FALSE
  )
  new_data_tbl <- copy_to(sc, new_data, overwrite = TRUE)

  ml_transform(spark_model, new_data_tbl) %>%
    dplyr::pull(prediction)
}

```

We can then initialize the service by executing the following:

```

service <- callr::r_bg(function() {
  p <- plumber::plumb("plumber/spark-plumber.R")
  p$run(port = 8000)
})

```

This starts the web service locally, and then we can query the service with new data to be scored; however, you might need to wait a few seconds for the Spark service to initialize:

```

httr::content(httr::POST(
  "http://127.0.0.1:8000/predict",
  body = '{"age": 42, "sex": "m", "drinks": "not at all",
    "drugs": "never", "essay_length": 99}'
))

```

```
[[1]]
```

```
[1] 0
```

This reply tell us that this particular profile is likely to not be unemployed, that is, employed. We can now terminate the `plumber` service by stopping the `callr` service:

```
service$interrupt()
```

If we were to time this operation (e.g., with `system.time()`), we see that the latency is on the order of hundreds of milliseconds, which might be appropriate for batch applications but is insufficient for real time. The main bottleneck is the serialization of the R `DataFrame` to a Spark `DataFrame` and back. Also, it requires an active Spark session, which is a heavy runtime requirement. To ameliorate these issues, we discuss next a deployment method more suitable for real-time deployment.

5.6.2 Real-Time Scoring

For real-time production, we want to keep dependencies as light as possible so we can target more platforms for deployment. We now show how we can use the `mleap` package, which provides an interface to the `MLeap` library, to serialize and serve Spark ML models. `MLeap` is open source (Apache License 2.0) and supports a wide range of, though not all, Spark ML transformers. At runtime, the only prerequisites for the environment are the Java Virtual Machine (JVM) and the `MLeap` runtime library. This avoids both the Spark binaries and expensive overhead in converting data to and from Spark `DataFrames`.

Since `mleap` is a `sparklyr` extension and an R package, first we need to install it from CRAN:

```
install.packages("mleap")
```

It then must be loaded when `spark_connect()` is called; so let's restart your R session, establish a new Spark connection,²⁰ and load the pipeline model that we previously saved:

```
library(sparklyr)
```

```
library(mleap)
```

```
sc <- spark_connect(master = "local", version = "2.3")
```

```
spark_model <- ml_load(sc, "spark_model")
```

The way we save a model to MLeap bundle format is very similar to saving a model using the Spark ML Pipelines API; the only additional argument is `sample_input`, which is a Spark DataFrame with schema that we expect new data to be scored to have:

```
sample_input <- data.frame(  
  sex = "m",  
  drinks = "not at all",  
  drugs = "never",  
  essay_length = 99,  
  age = 25,  
  stringsAsFactors = FALSE  
)
```

```
sample_input_tbl <- copy_to(sc, sample_input)
```

```
ml_write_bundle(spark_model, sample_input_tbl, "mleap_model.zip", overwrite = TRUE)
```

We can now deploy the artifact we just created, *mleap_model.zip*, in any device that runs Java and has the open source MLeap runtime dependencies, without needing Spark or R! In fact, we can go ahead and disconnect from Spark already:

```
spark_disconnect(sc)
```

Before we use this MLeap model, make sure the runtime dependencies are installed:

```
mleap::install_maven()  
mleap::install_mleap()
```

To test this model, we can create a new plumber API to expose it. The script *plumber/mleap-plumber.R* is very similar to the previous example:

```
library(mleap)

mleap_model <- mleap_load_bundle("mleap_model.zip")

#* @post /predict
score_spark <- function(age, sex, drinks, drugs, essay_length) {
  new_data <- data.frame(
    age = as.double(age),
    sex = sex,
    drinks = drinks,
    drugs = drugs,
    essay_length = as.double(essay_length),
    stringsAsFactors = FALSE
  )
  mleap_transform(mleap_model, new_data)$prediction
}
```

And the way we launch the service is exactly the same:

```
service <- callr::r_bg(function() {
  p <- plumber::plumb("plumber/mleap-plumber.R")
  p$run(port = 8000)
})
```

We can run the exact same code we did previously to test unemployment predictions in this new service:

```
httr::POST(
  "http://127.0.0.1:8000/predict",
  body = '{"age": 42, "sex": "m", "drinks": "not at all",
    "drugs": "never", "essay_length": 99}'
) %>%
  httr::content()
```

```
[[1]]
[1] 0
```

If we were to time this operation, we would see that the service now returns predictions in tens of milliseconds.

Let's stop this service and then wrap up this chapter:

```
service$interrupt()
```

5.7 Recap

In this chapter, we discussed Spark Pipelines, which is the engine behind the modeling functions introduced in [Chapter 4](#). You learned how to tidy up your predictive modeling workflows by organizing data processing and modeling algorithms into pipelines. You learned that pipelines also facilitate collaboration among members of a multilingual data science and engineering team by sharing a language-agnostic serialization format—you can export a Spark pipeline from R and let others reload your pipeline into Spark using Python or Scala, which allows them to collaborate without changing their language of choice.

You also learned how to deploy pipelines using `mleap`, a Java runtime that provides another path to productionize Spark models—you can export a pipeline and integrate it to Java-enabled environments without requiring the target environment to support Spark or R.

You probably noticed that some algorithms, especially the unsupervised learning kind, were slow, even for the `OKCupid` dataset, which can be loaded into memory. If we had access to a proper Spark cluster, we could spend more time modeling and less time waiting! Not only that, but we could use cluster resources to run broader hyperparameter-tuning jobs and process large datasets. To get there, [Chapter 6](#) presents what exactly a computing cluster is and explains the various options you can consider, like building your own or using cloud clusters on demand.

20. As of the writing of this book, MLeap does not support Spark 2.4.↩